VHDL Case Study

Designing a 64-bit multiplier

H. MATHIAS
C2N
UNIVERSITE PARIS-SACLAY

Objectives

Identify the best architecture according to the constraints of the specifications

- **Compare different architectures:**

  Design combinatorial or sequential architectures in different ways in order to try to optimize this or that point of the specifications – try to find the best compromise

- **Criteria to be optimized:**

  Ccomputation time: Quantitatively determine the critical path of combinatorial circuits – know the number of clock cycles and max clock frequency for sequential circuits

  Size (cost): Synthesize the circuits and see the necessary resources

  Power Consumption : Synthesize circuits and estimate consumption at operating frequency

  Time to market : minimize the development time of the synthesizable VHDL model

- **CAD tools used:**

  VHDL: adapt the code to obtain the desired architecture – to be validated in simulation (Modelsim)

  Logical synthesis: implementation on FPGA => use of Quartus (Altera)

  ASIC integration => use of CADENCE

Formalization of the problem to be solved

Develop a formal description of the behavior of the circuit to be designed

- **Multiplier principle (unsigned numbers) :**

4-bit example:

13 x 5 = 65

```
      1 1 0 1      A
    x 0 1 0 1      B
  -------------
      1 1 0 1      b0 = 1
    0 0 0 0        b1 = 0
  1 1 0 1          b2 = 1
0 0 0 0            b3 = 0
-------------
0 1 0 0 0 0 0 1
```

2 steps of ccomputation :

1) Generation of partial products:

if b_i=0, PPi=0 else PPi = A * 2^i

2) Addition of partial products

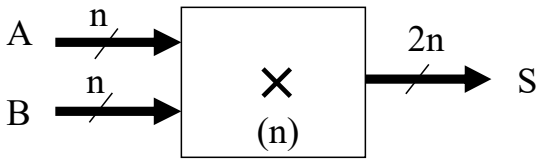
- **Computation algorithm:** →

```
sum = 0;
for i = 0 to n-1 do
    if bi = 0 then
        PP = 0;
    else
        PP = A * 2^i;
    end if;
    sum = sum + PP;
end for;
S = sum;
```

Description of the entity

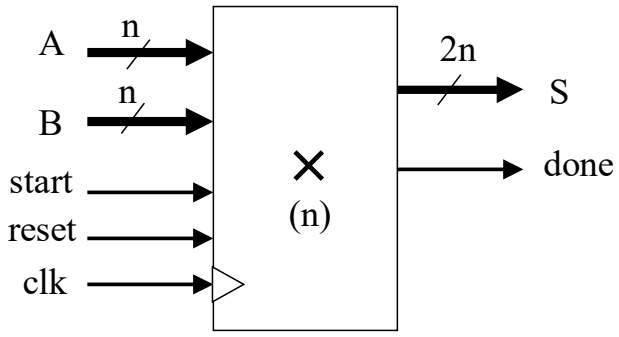
Creating a generic component: multiplier on n bits = > best future Time to Market

Combinatorial version




```
entity multiplier is
  generic(n : natural := 32);
  port( A, B : in std_logic_vector(n-1  downto 0);
        S : out std_logic_vector(2*n-1  downto 0)
  );
end entity multiplier;
```

Sequential version




```
entity mult_seq is
  generic(n : natural := 32);
  port( clk : in std_logic;
        reset  : in std_logic;
        start : in std_logic;
        A, B : in std_logic_vector(n-1 downto 0);
        S : out std_logic_vector(2*n-1 downto 0);
        done : out std_logic
  );
end entity mult_seq;
```

Combinatorial architectures

Architecture 1 : comb_hl

```
architecture comb_hl of multiplier is
begin

  S <= std_logic_vector(unsigned(A)*unsigned(B));

end architecture comb_hl;
```

Best possible Time to Market

Other criteria depend on the synthesizer and the target (FPGA, ASIC)

Combinatorial architectures

Architecture 2 : comb_proc

```
architecture comb_proc of multiplier is
begin
  PCalc : process(A,B) is
    variable PP : unsigned(2*n-1 downto 0); -- partial product
    variable sum : unsigned(2*n-1 downto 0); -- sum of partial products
    begin
      if B(0) = '0' then
        sum := to_unsigned(0,2*n);
      else
        sum := resize(unsigned(A), 2*n);
      end if;
      for i in 1 to n-1 loop
        if B(i) = '0' then
          PP := to_unsigned(0,2*n);
        else
          PP := resize(unsigned(A), 2*n);
        end if;
        sum := sum + (PP sll i);
      end loop;
      S <= std_logic_vector(sum);
    end process PCalc;
end architecture comb_proc;
```

Immediate translation of the algorithm using variables

Probably not optimal performance (final addition of partial products)

Combinatorial architectures

Architecture 3 : comb_proc2

```
architecture comb_proc2 of multiplier is
 begin
  PCalc : process(A,B) is
       variable PP : unsigned(n-1 downto 0); -- partial product
       variable bitB : unsigned(n-1 downto 0); -- current bit of B duplicated n times
       variable sum : unsigned(2*n-1 downto 0); -- sum of partial products
     begin
       bitB := (others => B(0));
       PP := unsigned(A) AND bitB;
       sum := resize(PP, 2*n);
       for i in 1 to n-1 loop
            bitB := (others => B(i));
            PP := unsigned(A) AND bitB;
            sum := sum + resize(PP sll i,2*n);
       end loop;
       S <= std_logic_vector(sum);
   end process PCalc;
end architecture comb_proc2;
```

Using logical ANDs to calculate partial products

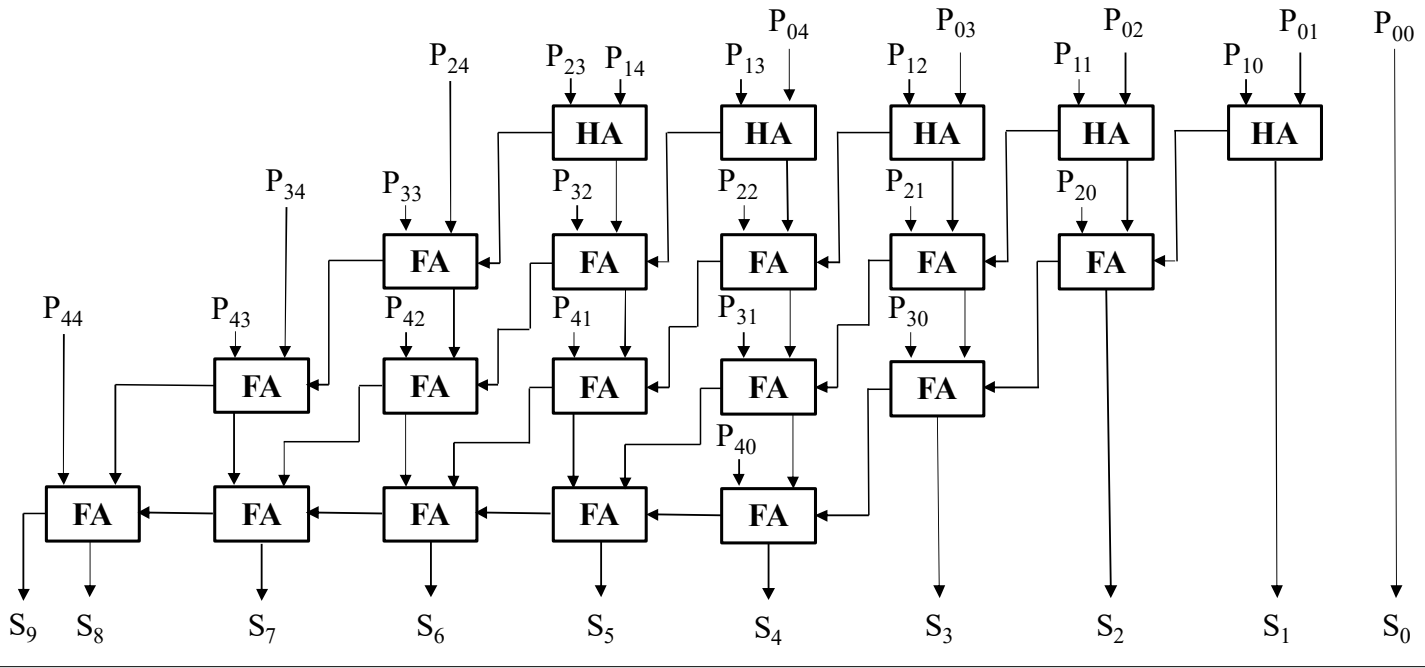
Probably not optimal performance (final addition of partial products)

Combinatorial architectures

Modeling a carry-save adder

$$
\begin{array}{rccccccccc}
 & & & & P_{04} & P_{03} & P_{02} & P_{01} & P_{00} & \\
+ & & & P_{14} & P_{13} & P_{12} & P_{11} & P_{10} & & \\
+ & & P_{24} & P_{23} & P_{22} & P_{21} & P_{20} & & & \\
+ & P_{34} & P_{33} & P_{32} & P_{31} & P_{30} & & & & \\
+ & P_{44} & P_{43} & P_{42} & P_{41} & P_{40} & & & & \\
\hline
 & S_9 & S_8 & S_7 & S_6 & S_5 & S_4 & S_3 & S_2 & S_1 & S_0
\end{array}
$$

Parallelized multioperand addition for partial products



Combinatorial architectures

Architecture 4 : comb_proc3

Computation divided into two independent processes:

Pcalc_PP : Calculates all partial products

Pcalc_sumPP_csa : calculates the sum of PP using a CSA (based on FOR loops)

```
architecture comb_proc3 of multiplier is
    type tableau is array (n-1 downto 0) of std_logic_vector(n-1 downto 0);
    signal PP : tableau; -- all the partial products
 begin
 PCalc_PP : process(A,B) is
    variable PPint : tableau; -- array of partial products
    variable bitB : std_logic_vector(n-1 downto 0); -- current bit of B duplicated n times
  begin
    for i in 0 to n-1 loop
       bitB := (others => B(i));
       PPint(i) := A AND bitB;
    end loop;
    PP <= PPint;
  end process PCalc_PP;
```

```
PCalc_sumPP_csa : process(PP) is
    variable Sint : std_logic_vector(2*n-1 downto 0);
    variable Rint,Cint : std_logic_vector(n-1 downto 0);-- internal results and carry-out
  begin
    Rint := PP(0);
    Cint := (others => '0');
    Sint(0) := Rint(0);
    for i in 1 to n-1 loop
      if (i /= (n-1)) then
        for j in 0 to n-1 loop
          if ((i+j) <= (n-1)) then
            Rint(j) := Rint(j+1) XOR Cint(j) XOR PP(i)(j);
            Cint(j) := (Rint(j+1) AND PP(i)(j)) OR (Cint(j) AND (Rint(j+1) OR PP(i)(j)));
          elsif (j/=(n-1)) then
            Rint(j) := Rint(j+1) XOR Cint(j) XOR PP(i+1)(j-1);
            Cint(j) := (Rint(j+1) AND PP(i+1)(j-1)) OR (Cint(j) AND (Rint(j+1) OR PP(i+1)(j-1)));
          else
            Rint(j) := PP(i)(j) XOR Cint(j) XOR PP(i+1)(j-1);
            Cint(j) := (PP(i)(j) AND PP(i+1)(j-1)) OR (Cint(j) AND (PP(i)(j) OR PP(i+1)(j-1)));
          end if;
        end loop;
        Sint(i):= Rint(0);
      else
        for j in 0 to n-1 loop
          if ((i+j) <= (n-1)) then
            Rint(j) := Rint(j+1) XOR Cint(j) XOR PP(i)(j);
            Cint(j) := (Rint(j+1) AND PP(i)(j)) OR (Cint(j) AND (Rint(j+1) OR PP(i)(j)));
          elsif (j/=(n-1)) then
            Rint(j) := Rint(j+1) XOR Cint(j) XOR Cint(j-1);
            Cint(j) := (Rint(j+1) AND Cint(j)) OR (Cint(j-1) AND (Rint(j+1) OR Cint(j)));
          else
            Rint(j) := PP(i)(j) XOR Cint(j) XOR Cint(j-1);
            Cint(j) := (PP(i)(j) AND Cint(j)) OR (Cint(j-1) AND (PP(i)(j) OR Cint(j)));
          end if;
        end loop;
        Sint(2*n-2 downto n-1):= Rint;
        Sint(2*n-1) := Cint(n-1);
      end if;
    end loop;
    S <= Sint;
  end process PCalc_sumPP;
end architecture comb_proc3;
```

Combinatorial architectures

Architecture 5 : comb_proc4

Computation divided into two independent processes:

Pcalc_PP: Calculates all partial products

Calculating the sum of PP using a generate-based csa

```
architecture comb_proc4 of multiplier is
   type tableau is array (n-1 downto 0) of std_logic_vector(n-1 downto 0);
   signal PP, R, C : tableau; -- all the partial products, the internal results and carry-out
 begin
 PCalc_PP : process(A,B) is
    variable PPint : tableau; -- array of partial products
    variable bitB : std_logic_vector(n-1 downto 0); -- current bit of B duplicated n times
  begin
    for i in 0 to n-1 loop
       bitB := (others => B(i));
       PPint(i) := A AND bitB;
    end loop;
    PP <= PPint;
  end process PCalc_PP;
```

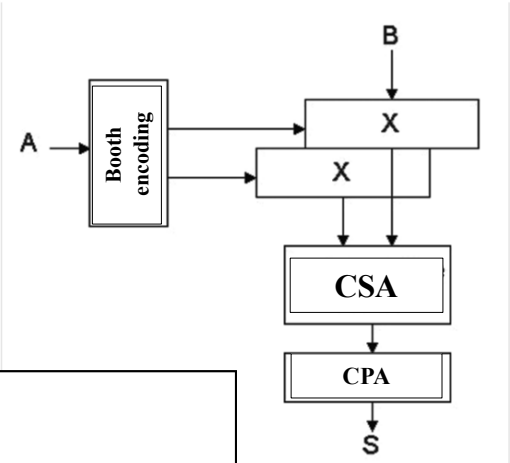
```
                R(0) <= PP(0);
                C(0) <= (others => '0');
                S(0) <= R(0)(0);
      firstPPsum: for j in 0 to n-2 generate
                    R(1)(j) <= R(0)(j+1) XOR C(0)(j) XOR PP(1)(j);
                    C(1)(j) <= (R(0)(j+1) AND PP(1)(j)) OR (C(0)(j) AND (R(0)(j+1) OR PP(1)(j)));
                  end generate;
                R(1)(n-1) <= PP(1)(n-1) XOR C(0)(n-1) XOR PP(2)(n-2);
                C(1)(n-1) <= (PP(1)(n-1) AND PP(2)(n-2)) OR (C(0)(n-1) AND (PP(1)(n-1) OR PP(2)(n-2)));
                S(1) <= R(1)(0);
         PPSum : for i in 2 to n-2 generate
       firstpartPPsum : for j in 0 to n-i-1 generate
                            R(i)(j) <= R(i-1)(j+1) XOR C(i-1)(j) XOR PP(i)(j);
                          C(i)(j) <= (R(i-1)(j+1) AND PP(i)(j)) OR (C(i-1)(j) AND (R(i-1)(j+1) OR PP(i)(j)));
                        end generate;
    secondpartPPsum : for j in n-i to n-2 generate
                            R(i)(j) <= R(i-1)(j+1) XOR C(i-1)(j) XOR PP(i+1)(j-1);
                            C(i)(j) <= (R(i-1)(j+1) AND PP(i+1)(j-1)) OR (C(i-1)(j) AND (R(i-1)(j+1) OR PP(i+1)(j-1)));
                        end generate;
                        R(i)(n-1) <= PP(i)(n-1) XOR C(i-1)(n-1) XOR PP(i+1)(n-2);
                        C(i)(n-1) <= (PP(i)(n-1) AND PP(i+1)(n-2)) OR (C(i-1)(n-1) AND (PP(i)(n-1) OR PP(i+1)(n-2)));
                        S(i) <= R(i)(0);
                end generate;
                R(n-1)(0) <= R(n-2)(1) XOR C(n-2)(0) XOR PP(n-1)(0);
                C(n-1)(0) <= (R(n-2)(1) AND PP(n-1)(0)) OR (C(n-2)(0) AND (R(n-2)(1) OR PP(n-1)(0)));
     LastPPsum : for j in 1 to n-2 generate
                      R(n-1)(j) <= R(n-2)(j+1) XOR C(n-2)(j) XOR C(n-1)(j-1);
                      C(n-1)(j) <= (R(n-2)(j+1) AND C(n-2)(j)) OR (C(n-1)(j-1) AND (R(n-2)(j+1) OR C(n-2)(j)));
                end generate;
                R(n-1)(n-1) <= PP(n-1)(n-1) XOR C(n-2)(n-1) XOR C(n-1)(n-2);
                C(n-1)(n-1) <= (PP(n-1)(n-1) AND C(n-2)(n-1)) OR (C(n-1)(n-2) AND (PP(n-1)(n-1) OR C(n-2)(n-1)));
                S(2*n-2 downto n-1)<= R(n-1);
                S(2*n-1) <= C(n-1)(n-1);
end architecture comb_proc4;
```

Combinatorial architectures

Example of a Booth multiplier (signed numbers) - formalization change

$$\mathbf{P} = \sum_{\mathbf{i=0}}^{\mathbf{n/2-1}} \mathbf{P_i 2^{2i}}$$

$$A = \sum_{i=0}^{n/2-1} \left(a_{2i-1} + a_{2i} - 2a_{2i+1}\right)2^{2i}; a_{-1} = 0$$



a_{2i+1}	a_{2i}	a_{2i-1}	P_i
0	0	0	0
0	0	1	+B
0	1	0	+B
0	1	1	+2B
1	0	0	-2B
1	0	1	-B
1	1	0	-B
1	1	1	0

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity multiplier is
  generic(n : natural := 32);
  port(A,B : in SIGNED(n-1 downto 0);
       S: out SIGNED(2*n-1 downto 0));
end multiplier;
```

Architecture 6: booth

```
architecture booth of multiplier is
type tableau is array (n/2-1 downto 0) of signed(2*n-1 downto 0);
signal PP : tableau;

begin
gene_pp: process(A,B)
  variable sel : signed(2 downto 0);
  variable PP_int : tableau;
  begin
    sel(0) := '0';
    sel(2 downto 1) := signed(A(1 downto 0));
    for i in 0 to n/2-1 loop
       case sel is
          when "000" => PP_int(i) := (others => '0');
          when "001" => PP_int(i) := resize(signed(B) sll (2*i),2*n);
          when "010" => PP_int(i) := resize(signed(B) sll (2*i),2*n);
          when "011" => PP_int(i) := resize((2*signed(B)) sll (2*i),2*n);
          when "100" => PP_int(i) := resize((-2*signed(B)) sll (2*i),2*n);
          when "101" => PP_int(i) := resize((-signed(B)) sll (2*i),2*n);
          when "110" => PP_int(i) := resize((-signed(B)) sll (2*i),2*n);
          when "111" => PP_int(i) := (others => '0');
          when others => PP_int(i) := (others => 'X');
       end case;
       if i /= (n/2-1) then
            sel := signed(A(2*i+3 downto 2*i+1));
       end if;
    end loop;
    PP <= PP_int;
end process gene_pp;
```

Combinatorial architectures

Architecture 6 : booth (Ctd)

```
addition: process(PP)
   variable result : signed(2*n-1 downto 0);
  begin
   result := (others => '0');
   for i in PP'RANGE loop
     result := result + PP(i);
   end loop;
   S <= std_logic_vector(result);
  end process addition;

end booth;
```

Note : As for the comb_proc architecture, it would be more efficient here in terms of performance to use a Carry Save Adder (multi-operator adder) to sum the partial products

Sequential architectures

Possible advantage: minimize material resources at the cost of execution in several clock cycles

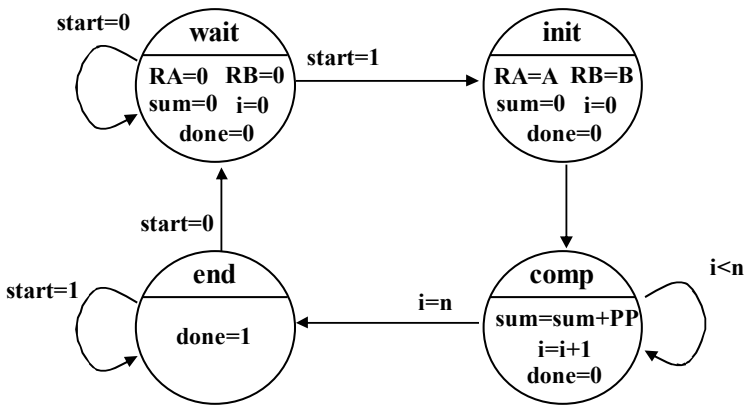
Translation of the algorithm as a finite state machine

To be able to implement the algorithm as a sequential circuit, it is necessary to translate it into a form where calculations progress as a result of successive clock cycles. It is necessary to identify in the algorithm the steps to be performed for each clock cycle, which is equivalent to translating the algorithm into a finite state machine.

In addition to the algorithm, take into account the signals that are used to control the circuit (start and done here: start goes to 1 to start the calculation and remains at 1 until the result has been retrieved; as soon as the calculation is complete and the result is available, done goes to 1; start goes back to 0 to be able to start a new calculation).

Constraint: 1 max addition / clock cycle

```
sum = 0;                          CC 1
for i = 0 to n-1 do
      if b_i = 0 then
         PP = 0;                  CC  2
      else                        to n+1
         PP = A * 2^i;
      end if;
      sum = sum + PP;
end for;                          CC
S = sum;                          n+2
```



Archi 1 - seq_proc : Calculation performed in n+2 CC – 2 adders needed

```
architecture seq_proc of mult_seq is
   signal RA, RB : unsigned(n-1 downto 0);
   signal sum : unsigned(2*n-1 downto 0); -- sum of partial products
   signal i : natural range 0 to n-1;
   type state is (wait_state, init_state, comp_state, end_state);
   signal curr_state : state;
begin

 S <= std_logic_vector(sum);

 PCalc : process(clk,reset) is
     variable PP : unsigned(n-1 downto 0); -- partial product
  begin
    if reset = '1' then
       curr_state <= wait_state;
    elsif clk'event and clk='1' then
       case curr_state is
         when wait_state => done <= '0';
                            RA <= to_unsigned(0,n);
                            RB <= to_unsigned(0,n);
                            sum <= to_unsigned(0,2*n);
                            i <= 0;
                           PP := to_unsigned(0,n);
                           if start = '1' then
                               curr_state <= init_state;
                           else
                               curr_state <= wait_state;
                           end if;
         when init_state => done <= '0';
                           RA <= unsigned(A);
                           RB <= unsigned(B);
                           sum <= to_unsigned(0,2*n);
                           i <= 0;
                           curr_state <= comp_state;
         when comp_state => done <= '0';
                           if RB(i) = '0' then
                                PP := to_unsigned(0,n);
                           else
                                PP := unsigned(RA);
                           end if;
                           sum <= sum + resize((PP sll i),2*n);
                           if i=n-1 then
                                i <= 0;
                                curr_state <= end_state;
                           else
                                i <= i+1;
                                curr_state <= comp_state;
                           end if;
         when end_state =>  done <= '1';
                           if start = '1' then
                             curr_state <= end_state;
                           else
                             curr_state <= wait_state;
                           end if;
         when others =>     done <= '0';
                            RA <= to_unsigned(0,n);
                            RB <= to_unsigned(0,n);
                            sum <= to_unsigned(0,2*n);
                            i <= 0;
                           PP := to_unsigned(0,n);
                           curr_state <= wait_state;
       end case;
     end if;
  end process PCalc;

end architecture seq_proc;
```

Architecture 1 : seq_proc

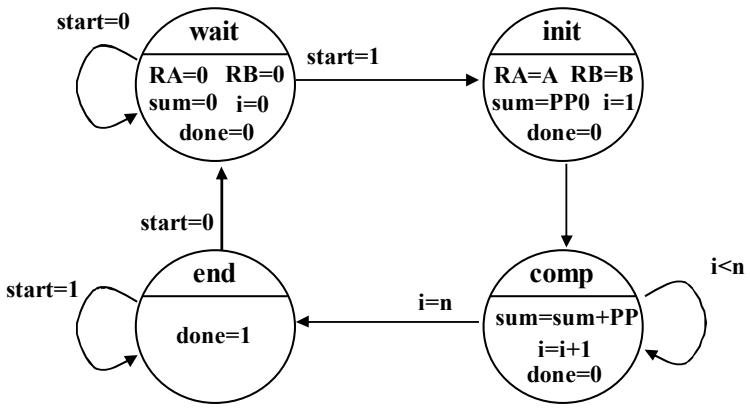
Sequential architectures

Translation of the algorithm as a finite state machine

Possible improvement by changing the formulation: rewriting the algorithm

Constraint: 1 max addition / clock cycle

```
sum = PP0;                          CC 1
for i = 1 to n-1 do
      if b_i = 0 then
         PP = 0;                    CC
      else                          2 to n
         PP = A * 2^i;
      end if;
      sum = sum + PP;
end for;                            CC
S = sum;                            n+1
```



Archi 2 - seq_proc2 : Calculation made in n + 1 CC – 2 adders needed

Saving time a priori without increasing size

```
architecture seq_proc2 of mult_seq is
   signal RA, RB : unsigned(n-1 downto 0);
   signal sum : unsigned(2*n-1 downto 0); -- sum of partial products
   signal i : natural range 0 to n-1;
   type state is (wait_state, init_state, comp_state, end_state);
   signal curr_state : state;
begin
  S <= std_logic_vector(sum);
  PCalc : process(clk,reset) is
      variable PP : unsigned(n-1 downto 0); -- partial product
   begin
     if reset = '1' then
       curr_state <= wait_state;
     elsif clk'event and clk='1' then
       case curr_state is
         when wait_state => done <= '0';
                            RA <= to_unsigned(0,n);
                            RB <= to_unsigned(0,n);
                            sum <= to_unsigned(0,2*n);
                            i <= 1;
                            PP := to_unsigned(0,n);
                            if start = '1' then
                                curr_state <= init_state;
                            else
                                curr_state <= wait_state;
                            end if;
         when init_state => done <= '0';
                            RA <= unsigned(A);
                            RB <= unsigned(B);
                            if B(0) = '0' then
                                sum := to_unsigned(0,2*n);
                            else
                                sum := resize(unsigned(A), 2*n);
                            end if;
                            i <= 1;
                            curr_state <= comp_state;
         when comp_state => done <= '0';
                            if RB(i) = '0' then
                                PP := to_unsigned(0,n);
                            else
                                PP := unsigned(RA);
                            end if;
                            sum <= sum + resize((PP sll i),2*n);
                            if i=n-1 then
                                i <= 0;
                                curr_state <= end_state;
                            else
                                 i <= i+1;
                                 curr_state <= comp_state;
                            end if;
         when end_state =>  done <= '1';
                            if start = '1' then
                                curr_state <= end_state;
                            else
                                curr_state <= wait_state;
                            end if;
         when others =>     done <= '0';
                            RA <= to_unsigned(0,n);
                            RB <= to_unsigned(0,n);
                            sum <= to_unsigned(0,2*n);
                            i <= 1;
                            PP := to_unsigned(0,n);
                            curr_state <= wait_state;
       end case;
     end if;
  end process PCalc;

end architecture seq_proc2;
```

Architecture 2 : seq_proc2

```
architecture seq_proc3 of mult_seq is
   signal RA, RB : unsigned(n-1 downto 0);
   signal sum : unsigned(2*n-1 downto 0); -- sum of part. products
   signal i : natural range 0 to n-1;
   type state is (wait_state, init_state, comp_state, end_state);
   signal curr_state : state;
begin
  S <= std_logic_vector(sum);
  PCalc : process(clk,reset) is
      variable PP : unsigned(n-1 downto 0); -- partial product
   begin
     if reset = '1' then
       curr_state <= wait_state;
     elsif clk'event and clk='1' then
       case curr_state is
         when wait_state => done <= '0';
                            RA <= to_unsigned(0,n);
                            RB <= to_unsigned(0,n);
                            sum <= to_unsigned(0,2*n);
                            i <= 1;
                            PP := to_unsigned(0,n);
                            if start = '1' then
                                curr_state <= init_state;
                            else
                                curr_state <= wait_state;
                            end if;
         when init_state => done <= '0';
                            RA <= unsigned(A);
                            RB <= unsigned(B);
                            bitB := (others => B(0));
                            PP := unsigned(A) AND bitB;
                            sum<= resize(PP, 2*n);
                            i <= 1;
                            curr_state <= comp_state;
         when comp_state => done <= '0';
                            bitB := (others => B(i));
                            PP := unsigned(A) AND bitB;
                            sum <= sum + resize(PP sll i,2*n);
                            if i=n-1 then
                                i <= 0;
                                curr_state <= end_state;
                            else
                                 i <= i+1;
                                 curr_state <= comp_state;
                            end if;
         when end_state =>  done <= '1';
                            if start = '1' then
                                curr_state <= end_state;
                            else
                                curr_state <= wait_state;
                            end if;
         when others =>     done <= '0';
                            RA <= to_unsigned(0,n);
                            RB <= to_unsigned(0,n);
                            sum <= to_unsigned(0,2*n);
                            i <= 1;
                            PP := to_unsigned(0,n);
                            curr_state <= wait_state;
       end case;
     end if;
  end process PCalc;

end architecture seq_proc3;
```

Architecture 3 : seq_proc3

PP computation using AND gates

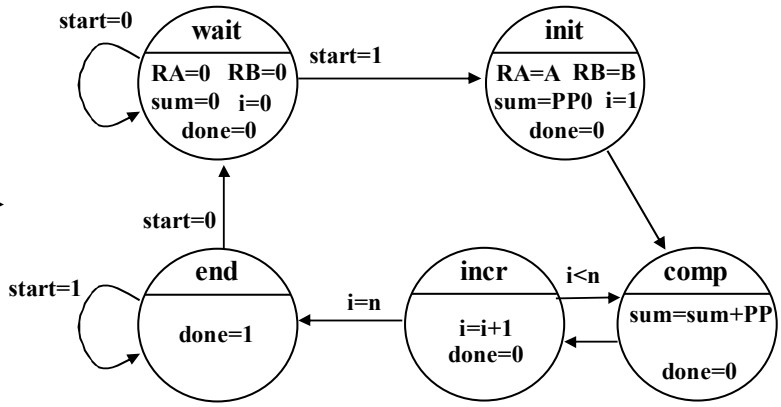
Sequential architectures

Translation of the algorithm as a finite state machine

Reduce size by changing the Finite State machine

Constraint: 1 max addition / clock cycle

```
sum = PP0;                          CC 1
for i = 1 to n-1 do
    if b_i = 0 then
        PP = 0;                     CC
    else                            2 to n
        PP = A * 2^i;
    end if;
    sum = sum + PP;
end for;                            CC
S = sum;                            n+1
```



Archi 4 - seq_proc4 : Calculation performed in 2n CC – 1 adder needed

Minimal size but longer calculation time

```
architecture seq_proc4 of mult_seq is
  signal RA, RB : unsigned(n-1 downto 0);
  signal sum : unsigned(2*n-1 downto 0); -- sum of part. products
  signal i : natural range 0 to n-1;
  type state is (wait_state, init_state, comp_state, incr_state, end_state);
  signal curr_state : state;
begin
 S <= std_logic_vector(sum);
 PCalc : process(clk,reset) is
    variable PP : unsigned(n-1 downto 0); -- partial product
  begin
   if reset = '1' then
     curr_state <= wait_state;
   elsif clk'event and clk='1' then
     case curr_state is
       when wait_state => done <= '0';
                          RA <= to_unsigned(0,n);
                          RB <= to_unsigned(0,n);
                          sum <= to_unsigned(0,2*n);
                          i <= 1;
                          PP := to_unsigned(0,n);
                          if start = '1' then
                              curr_state <= init_state;
                          else
                              curr_state <= wait_state;
                          end if;
       when init_state => done <= '0';
                          RA <= unsigned(A);
                          RB <= unsigned(B);
                          bitB := (others => B(0));
                          PP := unsigned(A) AND bitB;
                          sum<= resize(PP, 2*n);
                          i <= 1;
                          curr_state <= comp_state;
```

```
       when comp_state => done <= '0';
                          bitB := (others => B(i));
                          PP := unsigned(A) AND bitB;
                          sum <= sum + resize(PP sll i,2*n);
                          curr_state <= incr_state;
       when incr_state =>  if i=n-1 then
                              i <= 0;
                              curr_state <= end_state;
                          else
                              i <= i+1;
                              curr_state <= comp_state;
                          end if;
       when end_state =>  done <= '1';
                          if start = '1' then
                              curr_state <= end_state;
                          else
                              curr_state <= wait_state;
                          end if;
       when others =>     done <= '0';
                          RA <= to_unsigned(0,n);
                          RB <= to_unsigned(0,n);
                          sum <= to_unsigned(0,2*n);
                          i <= 1;
                          PP := to_unsigned(0,n);
                          curr_state <= wait_state;
     end case;
   end if;
 end process PCalc;

end architecture seq_proc4;
```

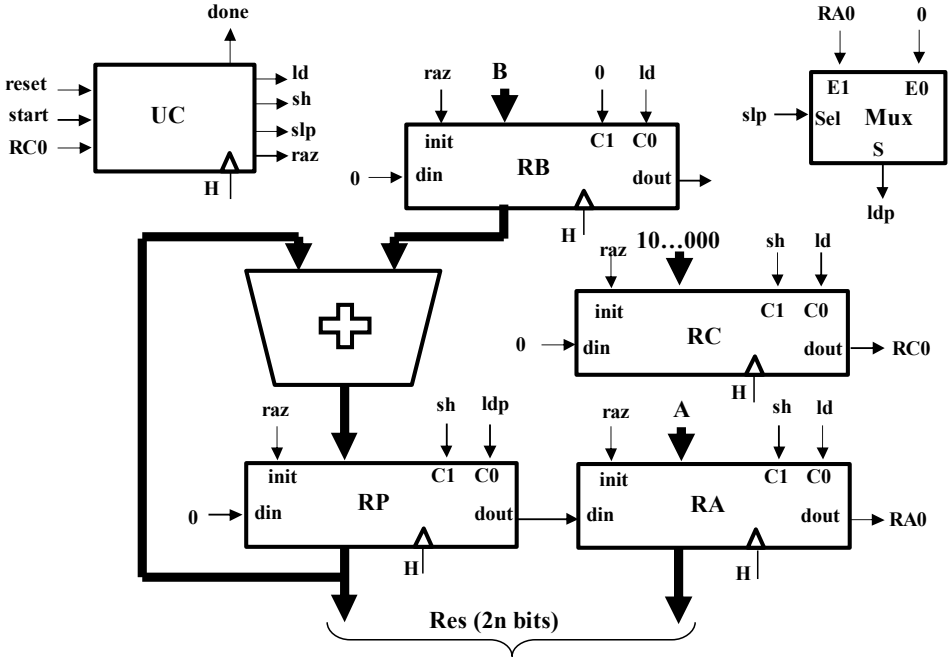
Architecture 4 : seq_proc4

Sequential architectures

Impose a specific architecture

In previous architectures, it is the interpretation of the code by the synthesizer that gives the circuit obtained. It is not necessarily the most optimal.

To reduce the size of the adder used, one solution may be to use the architecture below which uses offset registers. The advantage is that the adder now only has a size of n bits and not 2n as before. Bold links correspond to numbers of n bits except the RP input on n+1 bits.



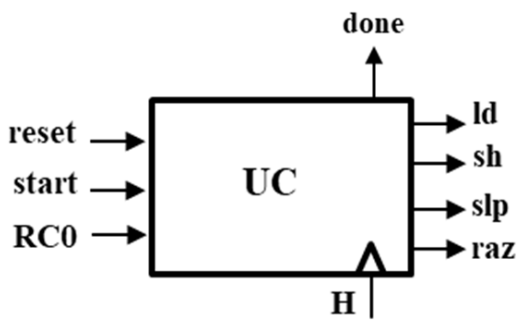
Sequential architectures

Impose a specific architecture

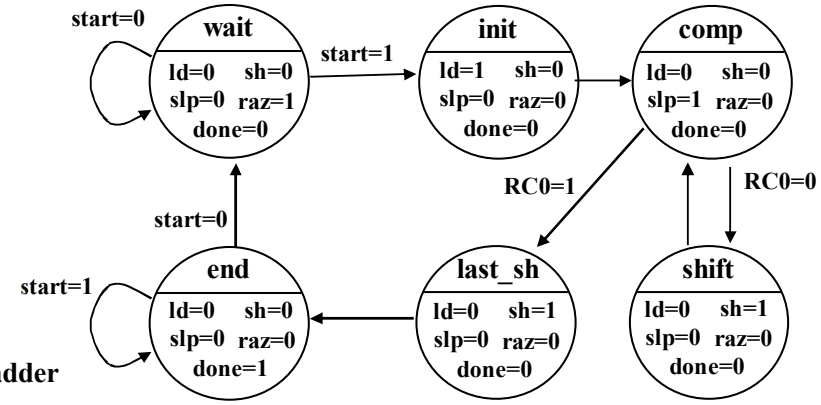
At the VHDL level, this requires a structural approach and therefore to develop all the blocks that may be needed. The control of the circuit obtained is therefore at the cost of a longer development time, especially since it is necessary to test all the blocks developed before using them.

In the previous example we have 3 blocks to develop for the operative part (data path): a shift register, a 2 to 1 multiplexer and an adder. In addition to this, it is necessary to develop the associated control unit and then assemble everything.

It is the control unit that will determine the number of clock strokes of the calculation. Be careful, its outputs ordering registers having the same clock as the control unit, it is necessary to ensure that they are modified as soon as the state changes (without waiting for the next clock stroke). This requires special modeling in VHDL based on **at least 2 processes** (Calculation and Memorization of the following state + Calculation of outputs).



Calculation performed in 2n+1 CC – 1 n-bit adder



```
entity control_unit is
port(reset, start, lb, clk : in STD_LOGIC;
       done, ld, sh, raz, selp : out STD_LOGIC);
end control_unit;

architecture proced of control_unit is
    type state is (wait_state, init_state, comp_state, shift_state,
                   last_shift_state, end_state);
    signal current_state, next_state : state;
begin
  CSM: process(clk, reset) -- current state memorization
    begin
      if (reset = '1') then
          current_state <= wait_state;
      elsif (clk'event and (clk = '1')) then
          current_state <= next_state;
      end if;
     end process CSM;
   OC: process(current_state) -- output computation
    begin
       case current_state is
          when wait_state => done <= '0';    ld <= '0';    sh <= '0';
                             raz <= '1';    selp <= '0';
          when init_state =>  done <= '0';    ld <= '1';    sh <= '0';
                             raz <= '0';     selp <= '0';
          when comp_state => done <= '0';   ld <= '0';   sh <= '0';
                              raz <= '0';   selp <= '1';
          when shift_state => done <= '0';     ld <= '0';   sh <= '1';
                              raz <= '0';    selp <= '0';
          when last_shift_state => done <= '0';  ld <= '0';  sh <= '1';
                                   raz <= '0';   selp <= '0';
          when end_state => done <= '1';   ld <= '0';   sh <= '0';
                            raz <= '0';    selp <= '0';
          when others => done <= '0';   ld <= '0';   sh <= '0';
                         raz <= '1';    selp <= '0';
        end case;
end process OC;
```

```
  NSC: process(current_state,start,lb) -- next state computation
    begin
      case current_state is
         when wait_state => if start = '1' then
                               next_state <= init_state;
                            else
                               next_state <= wait_state;
                            end if;
         when init_state =>   next_state <= comp_state;
         when comp_state => if lb = '1' then
                               next_state <= last_shift_state;
                            else
                               next_state <= shift_state;
                            end if;
         when shift_state =>  next_state <= comp_state;
         when last_shift_state =>  next_state <= end_state;
         when end_state =>  if start = '1' then
                               next_state <= end_state;
                            else
                               next_state <= wait_state;
                            end if;
         when others => next_state <= wait_state;
      end case;
  end process NSC;

end proced;
```

Control_unit block

```
entity REGLD_SR is -- Register with load and right-shift
    generic(nb_bits : natural := 32); -- stored data bit width
    Port ( clk, reset, load, shift : in std_logic;
           D: in std_logic_vector( nb_bits-1 downto 0);
           Din : in std_logic;  -- right shift MSB input
           Dout : out std_logic;  -- LSB output
           Q: out std_logic_vector(nb_bits-1 downto 0 )
    );
end REGLD_SR;

architecture proced of REGLD_SR is

signal Sint: std_logic_vector(nb_bits-1 downto 0 );

begin

 P1 : process(clk, reset)

   begin
    if reset='1' then
      Sint <= (others => '0');
    elsif clk'event and clk='1' then
      if load = '1' then
        Sint <= D;
      elsif shift = '1' then
        Sint <= Din & Sint(nb_bits-1 downto 1);
      end if;
    end if;
  end process P1;

 Q <= Sint;
 Dout <=  Sint(0);

end architecture proced;
```

```
entity adder is
    generic(nb_bits : natural:=32);
    port(A,B : in STD_LOGIC_VECTOR(nb_bits-1 downto 0);
         S : out STD_LOGIC_VECTOR(nb_bits-1 downto 0); -- result
         Cout : out STD_LOGIC);  -- carry out
end entity adder;

architecture dataflow of adder is
signal Sint : unsigned(nb_bits downto 0);
begin

Sint <= ('0' & unsigned(A))+('0' & unsigned(B));

S <= std_logic_vector(Sint(nb_bits-1 downto 0));
Cout <= Sint(nb_bits);

end architecture dataflow;
```

```
entity mux2_1 is
     port(I0,I1 : in std_logic;  --data inputs
          sel : in std_logic;  -- selection input
          S : out std_logic);
end entity mux2_1;

architecture proced of mux2_1 is
signal Sint : std_logic;
begin
   process(sel, I0, I1)
      begin
        if (sel = '0') then
            Sint <= I0;
        else
            Sint <= I1;
        end if;
    end process;
    S <= Sint;
end proced;
```

Regld_sr, adder and Mux2_1 blocks

```
architecture seq_det of mult_seq is
    signal sig_ld, sig_raz, sig_sh, sig_selp, sig_RB0, sig_RC0, zero, sig_ldp, sig_dop, sig_doa, sig_cout : std_logic;
    signal sig_add, sig_RP : std_logic_vector(n downto 0);
    signal sig_sum, sig_RA, sig_RB, sig_C, sig_RC : std_logic_vector(n-1 downto 0);
begin
   zero <= '0';
   sig_add <= sig_cout & sig_sum;
   sig_C(n-1) <= '1';
   sig_C(n-2 downto 0) <= ( others => '0');
   S <= sig_RP(n-1 downto 0) & sig_RB;
   MUX1: entity work.mux2_1(proced)
            port map(I0 => zero, I1 => sig_RB0, sel => sig_selp, S => sig_ldp);
   REGB: entity work.regld_sr(proced)
            generic map(nb_bits => n)
            port map(clk => clk, reset => sig_raz, load => sig_ld, D => B, Q => sig_RB, shift => sig_sh, din => sig_dop, dout => sig_RB0);
   REGA: entity work.regld_sr(proced)
            generic map(nb_bits => n)
            port map(clk => clk, reset => sig_raz, load => sig_ld, D => A, Q => sig_RA, shift => zero, din => zero, dout => sig_doa);
   REGC: entity work.regld_sr(proced)
            generic map(nb_bits => n)
            port map(clk => clk, reset => sig_raz, load => sig_ld, D => sig_C, Q => sig_RC, shift => sig_sh, din => zero, dout => sig_RC0);
   REGP: entity work.regld_sr(proced)
            generic map(nb_bits => n+1)
            port map(clk => clk, reset => sig_raz, load => sig_ldp, D => sig_add, Q => sig_RP, shift => sig_sh, din => zero, dout => sig_dop);
   ADDER: entity work.adder(dataflow)
              generic map(nb_bits => n)
              port map(A => sig_RA, B => sig_RP(n-1 downto 0), S => sig_sum, Cout => sig_cout);
   CU: entity work.control_unit(proced)
         port map(clk => clk,   reset => reset,
                  start => start,
                  lb => sig_RC0,
                  done => done,
                  ld => sig_ld,
                  sh => sig_sh,
                  raz => sig_raz,
                  selp => sig_selp);
end architecture seq_det;
```

Architecture 5 : seq_det

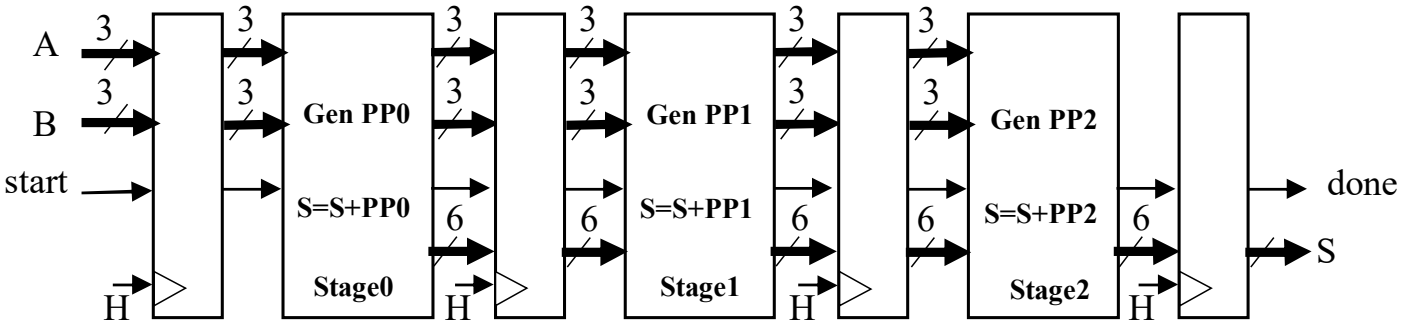
Sequential architectures

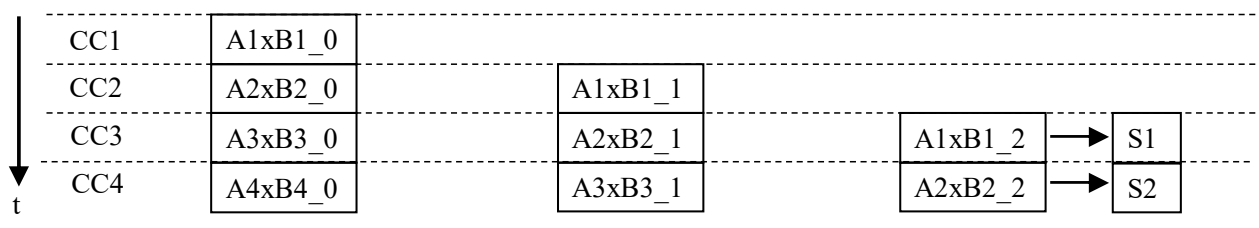
Pipeline architecture

Another interest of sequential architectures can be to significantly reduce computing times by parallelizing them, at the cost of the architecture size : this is the principle of the pipeline.

The idea is to divide the combinatorial circuit into N stages separated by registers. Each individual calculation will always be done in N successive steps (N clock cycles), but it can be arranged so that at a given moment each stage processes different data (N calculations are thus carried out in parallel). And with each new clock cycle, we get a new result as output, as if the calculation took only one clock cycle.

From the comb_proc architecture, we obtain the following pipeline version for the multiplier (case where n = N = 3 – the reset is not included for simplicity):





```
architecture pipeline of mult_seq is
   type array_sum is array (0 to n) of unsigned(2*n-1 downto 0);
   signal sum : array_sum;
   type array_AB is array (0 to n) of unsigned(n-1 downto 0);
   signal RA, RB : array_AB;
   signal start_array : std_logic_vector(n-1 downto 0);
begin
  PCalc : process(clk,reset) is
      type array_PP is array (0 to n-1) of unsigned(2*n-1 downto 0);
      variable PP : array_PP;
      variable BB : unsigned(n-1 downto 0);
   begin
     for i in 0 to n-1 loop
        BB := RB(i);
        if BB(i) = '0' then
           PP(i) := to_unsigned(0,2*n);
        else
           PP(i) := resize(unsigned(RA(i)), 2*n);
        end if;
        if reset = '1' then
           RA(i) <= to_unsigned(0,n);
           RB(i) <= to_unsigned(0,n);
           start_array(i) <= '0';
           sum(i) <= to_unsigned(0,2*n);
        elsif clk'event and clk='1' then
           if i=0 then
              RA(i) <= unsigned(A);
              RB(i) <= unsigned(B);
              start_array(i) <= start;
              sum(i) <= PP(i);
           else
              RA(i) <= RA(i-1);
              RB(i) <= RB(i-1);
              start_array(i) <= start_array(i-1);
              sum(i) <= sum(i-1) + (PP(i) sll i);
           end if;
        end if;
```

```
end loop;
   end process PCalc;

 REGS: process(clk, reset) is
  begin
    if reset ='1' then
       S <= (others => '0');
       done <= '0';
    elsif clk'event and clk='1' then
       S <= std_logic_vector(sum(n-1));
       done <= start_array(n-1);
    end if;
  end process REGS;

end architecture pipeline;
```

It is also possible to make a structural model using the generate statement

Validation in simulation

Testbench for combinatorial architectures

```
entity testbench is
end entity testbench;

architecture test_multiplier of testbench is
  constant nb_bits : natural := 16;
  signal sig_A, sig_B : std_logic_vector(nb_bits-1 downto 0);
  signal sig_S1, sig_S2, sig_S3, sig_S4, sig_S5, sig_S6 : std_logic_vector(2*nb_bits-1 downto 0);
begin

  DUT1: entity work.multiplier(comb_hl)
      generic map(n => nb_bits)
      port map(A => sig_A, B=> sig_B, S=> sig_S1);

  DUT2: entity work.multiplier(comb_proc)
      generic map(n => nb_bits)
      port map(A => sig_A, B=> sig_B, S=> sig_S2);

  DUT3: entity work.multiplier(comb_proc2)
      generic map(n => nb_bits)
      port map(A => sig_A, B=> sig_B, S=> sig_S3);

  DUT4: entity work.multiplier(comb_proc3)
      generic map(n => nb_bits)
      port map(A => sig_A, B=> sig_B, S=> sig_S4);

  DUT5: entity work.multiplier(comb_proc4)
      generic map(n => nb_bits)
      port map(A => sig_A, B=> sig_B, S=> sig_S5);

  DUT6: entity work.multiplier(booth)
      generic map(n => nb_bits)
      port map(A => sig_A, B=> sig_B, S=> sig_S6);

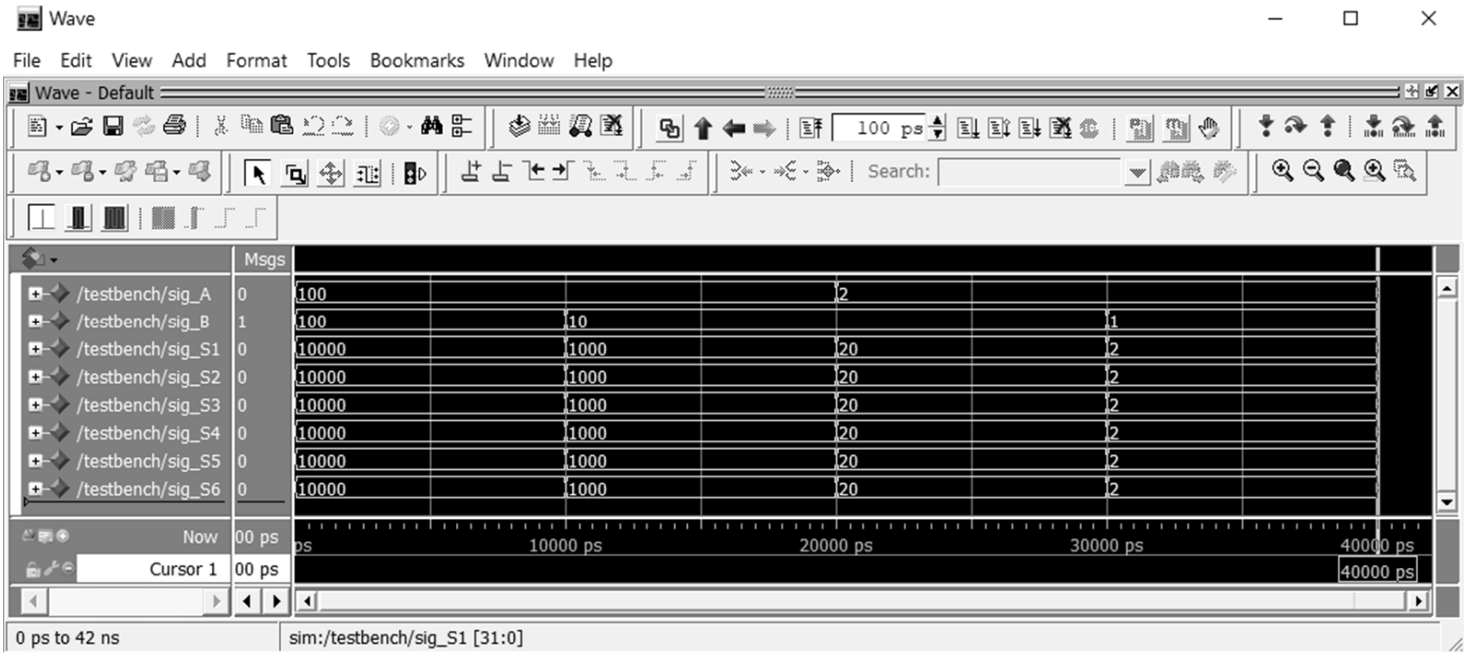
  sig_A <= std_logic_vector(to_unsigned(100,nb_bits)), std_logic_vector(to_unsigned(2,nb_bits)) after 20 ns, std_logic_vector(to_unsigned(0,nb_bits)) after 40 ns;
  sig_B <= std_logic_vector(to_unsigned(100,nb_bits)), std_logic_vector(to_unsigned(10,nb_bits)) after 10 ns, std_logic_vector(to_unsigned(1,nb_bits)) after 30 ns;

end architecture test_multiplier;
```

Validation in simulation

Results for combinatorial architectures

Simulation with Modelsim



All combinatorial architectures work correctly

```
architecture test_mult_seq of testbench is
  constant nb_bits : natural := 16;
  signal sig_A, sig_B : std_logic_vector(nb_bits-1 downto 0);
  signal sig_S1, sig_S2, sig_S3, sig_S4, sig_S5 : std_logic_vector(2*nb_bits-1 downto 0);
  signal sig_clk, sig_reset, sig_start, sig_done1, sig_done2, sig_done3, sig_done4, sig_done5 : std_logic;
Begin

  DUT1: entity work.mult_seq(seq_proc)
      generic map(n => nb_bits)
      port map(clk => sig_clk, reset => sig_reset, start => sig_start, A => sig_A, B=> sig_B, S=> sig_S1, done => sig_done1);
  DUT2: entity work.mult_seq(seq_proc2)
      generic map(n => nb_bits)
      port map(clk => sig_clk, reset => sig_reset, start => sig_start, A => sig_A, B=> sig_B, S=> sig_S2, done => sig_done2);
  DUT3: entity work.mult_seq(seq_proc3)
      generic map(n => nb_bits)
      port map(clk => sig_clk, reset => sig_reset, start => sig_start, A => sig_A, B=> sig_B, S=> sig_S3, done => sig_done3);
  DUT4: entity work.mult_seq(seq_proc4)
      generic map(n => nb_bits)
      port map(clk => sig_clk, reset => sig_reset, start => sig_start, A => sig_A, B=> sig_B, S=> sig_S4, done => sig_done4);
  DUT5: entity work.mult_seq(seq_det)
      generic map(n => nb_bits)
      port map(clk => sig_clk, reset => sig_reset, start => sig_start, A => sig_A, B=> sig_B, S=> sig_S5, done => sig_done5);

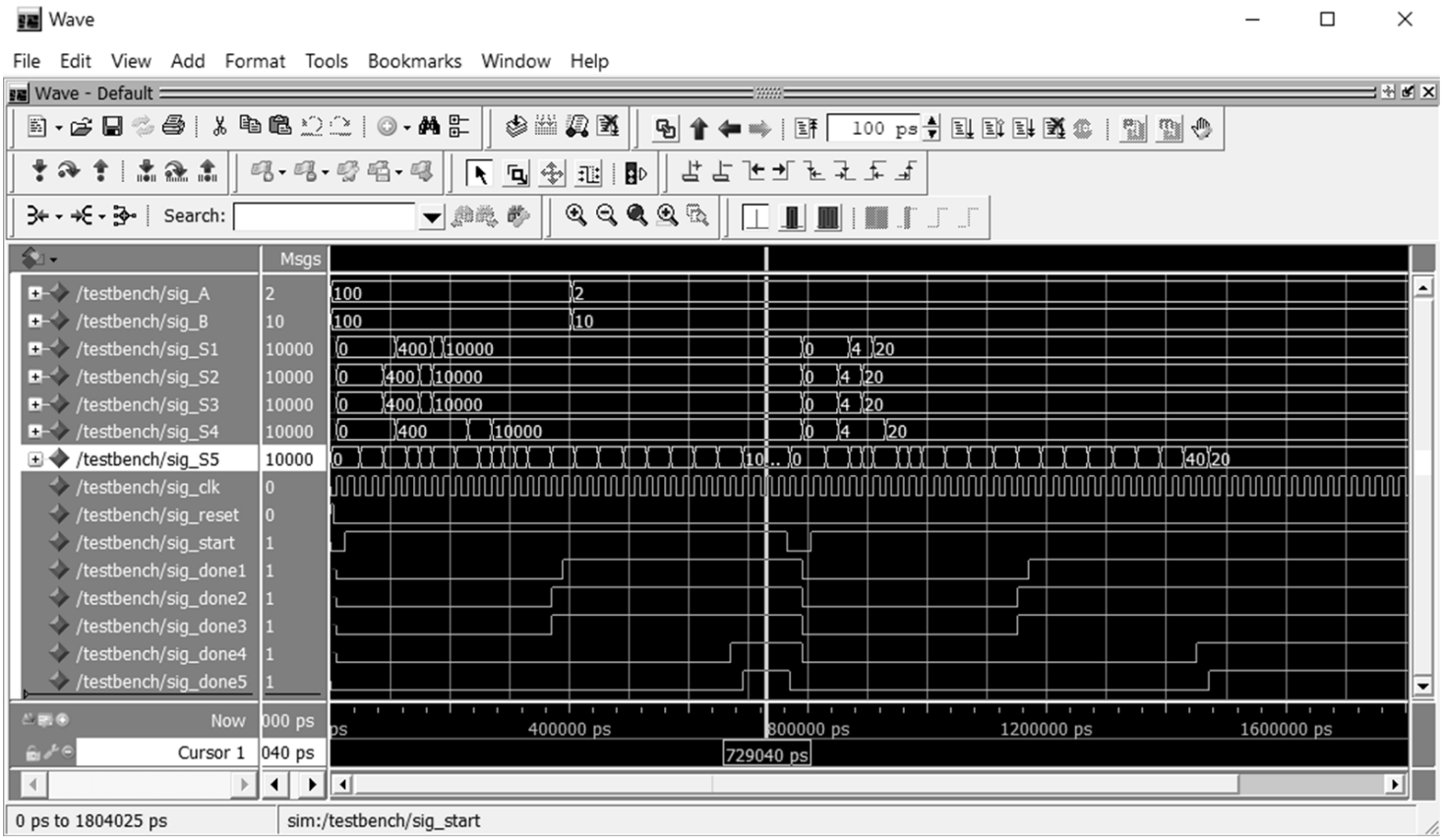
  sig_reset <= '1', '0' after 5 ns;
  sig_A <= std_logic_vector(to_unsigned(100,nb_bits)), std_logic_vector(to_unsigned(2,nb_bits)) after 405 ns;
  sig_B <= std_logic_vector(to_unsigned(100,nb_bits)), std_logic_vector(to_unsigned(10,nb_bits)) after 405 ns;
  sig_start <= '0', '1' after 25 ns, '0' after 765 ns, '1' after 805 ns;
  Pclk: process
    constant nb_periods : natural := 100;
    begin
       sig_clk <= '0';
       wait for 10 ns;
       for i in 1 to nb_periods loop
          sig_clk <= '1';
          wait for 10 ns;
          sig_clk <= '0';
          wait for 10 ns;
       end loop;
       wait;
  end process Pclk;

end architecture test_mult_seq;
```

Testbench for sequential architectures (non-pipeline)

Validation in simulation

Results for sequential architectures (non-pipeline)



All sequential architectures work well but with different computation times

```
architecture test_mult_pipeline of testbench is
  constant nb_bits : natural := 16;
  signal sig_A, sig_B : std_logic_vector(nb_bits-1 downto 0);
  signal sig_S : std_logic_vector(2*nb_bits-1 downto 0);
  signal sig_clk, sig_reset, sig_start, sig_done : std_logic;
begin

  DUT: entity work.mult_seq(pipeline)
      generic map(n => nb_bits)
      port map(clk => sig_clk, reset => sig_reset, start => sig_start, A => sig_A, B=> sig_B, S=> sig_S, done => sig_done);

  sig_reset <= '1', '0' after 5 ns;
  sig_A <= std_logic_vector(to_unsigned(100,nb_bits));
  sig_start <= '0', '1' after 25 ns;

  PB : process
    begin
       sig_B <= std_logic_vector(to_unsigned(0,nb_bits));
       wait for 35 ns;
       for i in 1 to 80 loop
          sig_B <= std_logic_vector(unsigned(sig_b)+i);
          wait for 20 ns;
       end loop;
       wait;
    end process PB;

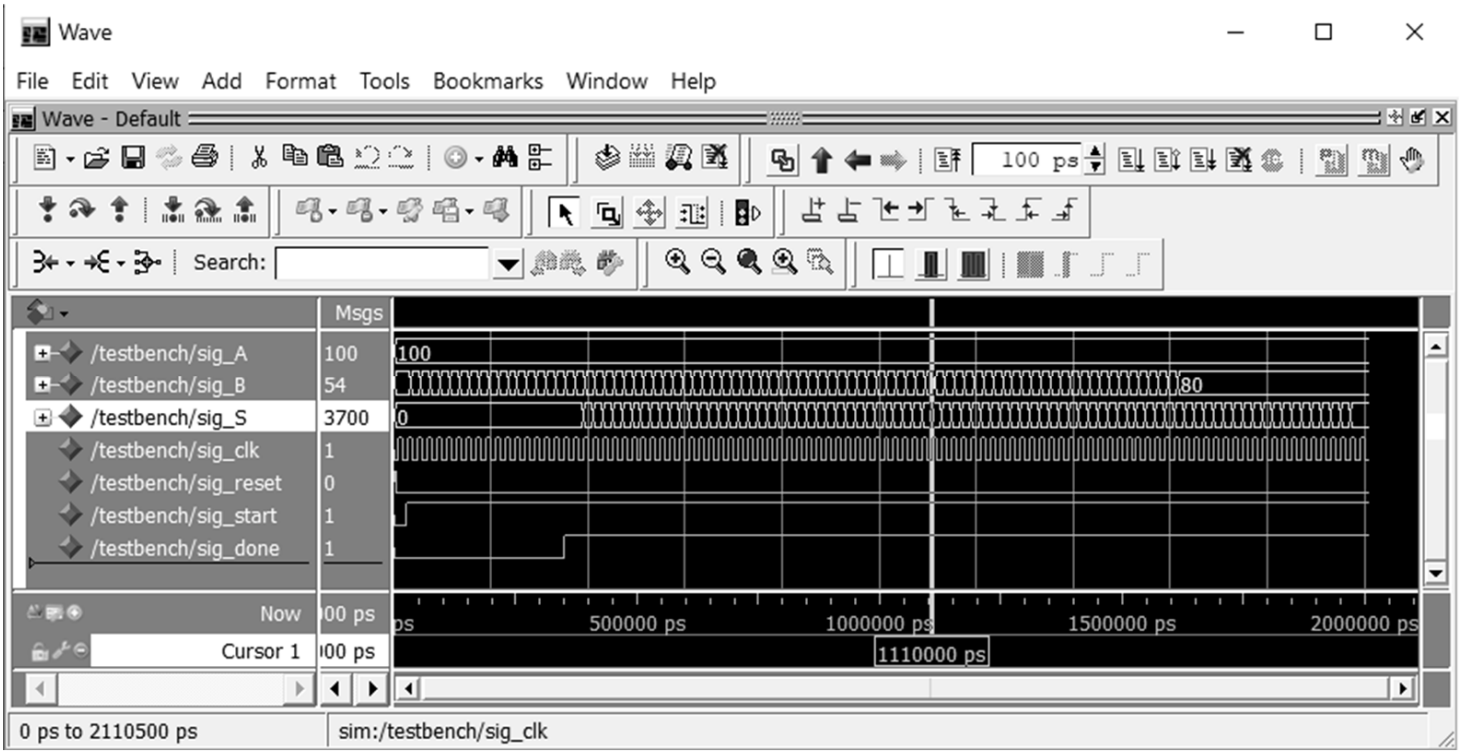

  Pclk: process
    constant nb_periods : natural := 100;
    begin
       sig_clk <= '0';
       wait for 10 ns;
       for i in 1 to nb_periods loop
          sig_clk <= '1';
          wait for 10 ns;
          sig_clk <= '0';
          wait for 10 ns;
       end loop;
       wait;
   end process Pclk;

end architecture test_mult_pipeline;
```

Testbench for pipeline architecture

Validation in simulation

Results for pipeline architecture



After a latency of n clock cycles for the first value, a new result is obtained at each clock cycle.

Summary review

Performance table – n=64 – Quartus (Cyclone 2 EP2C70F896C6 – 90nm ?)

Architecture	Nb CC	Fmax (MHz)	Nb LE and Reg	Min comp. time
Comb_hl	1	84.6	438 LE - 0 R	11.8 ns
Comb_proc	1	8.96	8190 LE - 0 R	111.6 ns
Comb_proc2	1	9.16	6112 LE - 0 R	109.2 ns
Comb_proc3 (csa)	1	11.44	12296 LE – 0 R	87.4 ns
Comb_proc4 (csa generate)	1	11.44	12296 LE – 0 R	87.4 ns
Booth	1	18.59	9373 LE - 0 R	53.8 ns
Seq_proc	66	57.7	872 LE - 267 R	1144 ns
Seq_proc2	65	56.7	873 LE - 267 R	1146 ns
Seq_proc3	65	57.8	715 LE - 139 R	1125 ns
Seq_proc4	128	58	715 LE - 140 R	2207 ns
Seq_det	129	140.3	314 LE - 266 R	919.5 ns
Pipeline	1	144.4	14872 LE - 14430 R	6.9 ns

Summary review

Performance table – n=64 – CADENCE (AMS 0.35µm)

Architecture	Nb CC	Fmax (MHz)	Area (µm²)	Min comp. time
Comb_hl	1	15.4	1112460	64.8 ns
Comb_proc	1	16.3	1505203	61.3 ns
Comb_proc2	1	29.1	772041	34.4 ns
Comb_proc3 (csa)	1	13.1	1511919	76.4 ns
Comb_proc4 (csa generate)	1	13.1	1511919	76.4 ns
Booth	1	16.2	1209801	61.9 ns
Seq_proc	66	149	438271	442 ns
Seq_proc2	65	153	474625	426 ns
Seq_proc3	65	152	427892	427 ns
Seq_proc4	128	150	397472	852 ns
Seq_det	129	288	290759	448 ns
Pipeline	1	237	10861848	4.2 ns